

Portfolio Question - Anomaly Detection Methods

Daniel Lawson University of Bristol

Portfolio 05.1.P (v4.0.0)

Portfolio Question 5.2.P2 - Anomaly Detection Methods

- ▶ What methods for anomaly detection are there? When should you use them?
- ▶ Give an explanation of some methods that are easily implemented in practical examples.

Multivariate: Mahalanobis distance

- ▶ Anomalies may be unremarkable in every single feature but **unusual in combination**.
- ▶ The Mahalanobis distance accounts for scale and correlation:

$$D^2(x) = (x - \mu)^T \Sigma^{-1} (x - \mu).$$

- ▶ If $x \sim \text{Normal}(\mu, \Sigma)$ in p dimensions, then $D^2(x) \sim \chi_p^2$, which gives a natural threshold.
- ▶ **But** plugging in the sample mean and covariance brings back masking.
 - ▶ **Minimum Covariance Determinant (MCD)**: estimate μ, Σ from the subset of h points whose covariance has the smallest determinant.
 - ▶ Implemented as `covMcd` (R, `robustbase`) and `EllipticEnvelope` (scikit-learn).

Density-based methods

- ▶ Use any density estimate $\hat{p}(x)$ and score $s(x) = -\log \hat{p}(x)$.
 - ▶ **Parametric**: a Gaussian, or a Gaussian Mixture Model (Lecture 04.2.1), which handles multimodal normal behaviour.
 - ▶ **Non-parametric**: Kernel Density Estimation.
- ▶ This is the most principled approach: “anomalous” means “improbable under the model”.
- ▶ Problems:
 - ▶ The density is only as good as the model, and a misspecified model makes structure look anomalous.
 - ▶ Density estimation is **hard in high dimensions** (curse of dimensionality).
 - ▶ The density depends on the parametrisation: a transformation of x changes which points have low density.

Distance-based methods

- ▶ **k-nearest-neighbour distance**: $s(x) = d(x, x_{(k)})$, the distance to the k th nearest neighbour.
 - ▶ No distributional assumption; depends on k , the metric, and **feature scaling**.
- ▶ This is a **global** method: one threshold applies everywhere.
- ▶ When normal data have regions of different density, a single distance threshold fails:
 - ▶ points in sparse clusters get flagged,
 - ▶ points just outside dense clusters get missed.

Local Outlier Factor (LOF)

- ▶ Compare a point's local density to that of its neighbours (Breunig et al., 2000).
- ▶ With $N_k(x)$ the k nearest neighbours of x :
 - ▶ **Reachability distance:** $r_k(x, o) = \max\{d_k(o), d(x, o)\}$, where $d_k(o)$ is o 's own k -NN distance,
 - ▶ **Local reachability density:**
$$\text{lrd}_k(x) = \left(\frac{1}{k} \sum_{o \in N_k(x)} r_k(x, o) \right)^{-1},$$
 - ▶ **LOF:** $\text{LOF}_k(x) = \frac{1}{k} \sum_{o \in N_k(x)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(x)}.$
- ▶ $\text{LOF} \approx 1$: as dense as its neighbours. $\text{LOF} \gg 1$: an outlier *relative to its neighbourhood*.
- ▶ LOF is a ratio with no fixed scale, so what counts as “large” depends on the dataset.

Clustering-based methods

- ▶ Clustering methods (Lecture 04.2) give anomaly scores almost for free:
 - ▶ **DBSCAN / HDBSCAN** label low-density points as **noise**,
 - ▶ **K-means / GMM**: distance to the nearest centroid, or low membership likelihood.
- ▶ A caution: these algorithms are optimised to find clusters, **not** anomalies.
 - ▶ Small clusters of anomalies may be given their own cluster and never flagged.
 - ▶ K-means centroids are themselves pulled towards outliers.

Reconstruction-based methods

- ▶ Assume normal data lie near a low-dimensional structure, and flag points that **don't fit it**.
- ▶ **PCA** (Lecture 04.1): project onto the top k components, $\hat{x} = V_k V_k^T x$ (centred data).
 - ▶ **Residual / SPE**: $\|x - \hat{x}\|^2$, how far the point is **off** the subspace,
 - ▶ **Hotelling's T^2** : $\sum_{j=1}^k z_j^2 / \lambda_j$, how extreme it is **within** the subspace.
 - ▶ These detect different things, and industrial process monitoring uses both.
- ▶ **Autoencoders** (Block 6) are the nonlinear version: train on normal data, score by reconstruction error.
 - ▶ Beware: a flexible enough model can reconstruct anomalies too.

Isolation Forest

- ▶ Instead of modelling normal, directly **isolate** each point (Liu, Ting & Zhou, 2008).
- ▶ Build random trees: choose a feature at random, then a split value uniformly between its min and max. Recurse until points are alone.
- ▶ **Anomalies are few and different, so they are isolated in few splits.**
- ▶ Score from the average path length $h(x)$ over trees, normalised by $c(n)$, the expected path length in a random binary tree:

$$s(x) = 2^{-\mathbb{E}[h(x)]/c(n)}.$$

- ▶ $s \rightarrow 1$: anomalous; $s \lesssim 0.5$: normal.
- ▶ Fast (subsampling, linear time), scales to high dimension, and has few tuning parameters. It is a strong default.
- ▶ Known weakness: axis-parallel splits produce artefacts in the score (addressed by *Extended Isolation Forest*).

Boundary methods

- ▶ Learn a region containing most of the normal data, and flag points outside it.
- ▶ **One-class SVM** (Schölkopf et al., 2001): separate the data from the origin in a kernel feature space.
 - ▶ $\nu \in (0, 1]$ bounds the fraction of training points treated as outliers.
- ▶ **Support Vector Data Description**: the smallest hypersphere in feature space enclosing the data.
- ▶ These connect to the SVMs of Lecture 01.2. They work well in the semi-supervised setting, but are sensitive to the kernel bandwidth and to contamination.

Practical difficulties

- ▶ **High dimensions:** distances concentrate, so “nearest” and “farthest” neighbours become similar. Anomalies may live in a few **subspaces** only.
- ▶ **Contaminated training data:** semi-supervised methods assume clean normal data, which is rarely true.
- ▶ **Mixed data types:** categorical features need their own distances or models.
- ▶ **Concept drift:** normal behaviour changes over time, so detectors must be refreshed, and the change itself may look anomalous.
- ▶ **Adversaries:** in fraud and security, anomalies adapt to avoid detection.
- ▶ **Interpretability:** the person who investigates a flagged case needs to know *why* it was flagged.

Discussion

- ▶ Lots of methods here - but which of them are commonly applied? Can you find applications?
 - ▶ What are they actually useful for?
 - ▶ How would you benchmark them?
- ▶ As an exercise: can you get the **sklearn examples** working on your data?
- ▶ **Review paper**